Build a Car Fuel-Efficiency Predictor

Group project · Plain Python

> **Your challenge:** predict how far a car goes on one gallon of fuel (its **MPG**) from three facts about the car: its mass, its engine size and its number of cylinders. Then show when your prediction should **not** be trusted.
>
> You already did this with the Rent Predictor. Now your group follows the same steps with new data, without the run-along.

What you hand in

	What	Details
1	Your notebook	**Student_Starter.ipynb** with tasks T01–T07 done. It must run from top to bottom with no errors.
2	Your answer sheet	**Car_Project_Report.docx** with every question (Q1–Q22) answered.
3	A 3-minute live demo	**On November 7.** Show one prediction, your test result and one mistake your model made.

Your files

File	What it is
Project_Guide.pdf	This guide: the steps, the seven tasks and the marking.
Student_Starter.ipynb	Your workspace. Open it in Jupyter. The data is already inside.
Car_Project_Report.docx	Your answer sheet. Open it in Word or Google Docs and type your answers in the boxes.

Rules

- Use plain Python only: lists, loops, functions and arithmetic. No pandas, NumPy or scikit-learn.
- Work in a group of 2 or 3 students. Every member must be able to explain the group's code.
- Do not change the data or move cars between groups.

How to work in the notebook

You see…	It means…
Shift+Enter	Run the selected cell. Always run cells in order, from the top.
NotImplementedError	Your task is here. Replace that line with your code.
assert …	A check on your work. Never delete it. If it fails, fix your code.
✏ YOUR ANSWER cell	Double-click it, type your answer, press Shift+Enter. There are three. Fill each one in **before** you run the next code cell, because they record your predictions.
Report Q…	Which answer-sheet question goes with this step.

Understand the data

Each row of data is one real car, written as a list of four numbers:

```
[weight_kg, engine_liters, cylinders, mpg]
```

Value	Meaning	Role
weight_kg	Mass of the car, in kilograms.	Input
engine_liters	Engine size (displacement), in litres. *Not* the size of the fuel tank.	Input
cylinders	Number of cylinders in the engine.	Input
mpg	Miles per US gallon. Higher MPG = the car goes farther on the same fuel.	**Target** (what we predict)

Example: [1589.388, 5.0308, 8, 18.0] is a 1,589 kg car with a 5.0 L, 8-cylinder engine that did 18 MPG. (It is record UCI_001, a 1970 Chevrolet Chevelle.)

Unit tip: 30 MPG is about 7.8 L/100 km. In MPG, higher is better. In L/100 km, lower is better.

Where the data comes from: 199 real cars, model years 1970–1982, from the UCI Auto MPG dataset. These are old cars, so this model is a learning exercise, not advice about modern cars.

Three groups of cars

Group	Cars	What it is for
Training	119	The model learns its numbers from these cars.
Validation	40	You use these to choose the best model.
Test	40	Used **once**, at the end, to score the model you chose.

Two golden rules

1. **Choose before you test.** Do not run the test cell (C12) until you have chosen your model and written your choice down (C11). If you look at the test score and then change your model, the test score stops meaning anything.
2. **Keep the hard cars.** Never delete a difficult example to get a better score.

Scaling the inputs

Big numbers like 1,589 make training unstable, so every input is divided by a fixed number. Use the **same** divisors for every group and for new cars:

```
x1 = weight_kg / 1000
x2 = engine_liters / 5
x3 = cylinders / 8
```

MPG is not scaled. It stays in MPG.

Your seven coding tasks

Each task replaces one NotImplementedError. The last column is the check that must pass.

Task	Cell	Write this	Must pass
T01 **make_features**	C04	Scale the inputs (x1, x2, x3 above). Mode "mass" returns [x1]. Mode "all" returns [x1, x2, x3]. Never put MPG in this list.	[1000, 2.5, 4, 999] gives [1.0, 0.5, 0.5]
T02 **predict**	C05	Start with the bias. For each position j, add features[j] × weights[j]. Return the total.	Features [1, 0.5, 0.5], weights [−10, −2, −4], bias 50 gives 37
T03 **mean_target**	C06	Add up the targets, then divide by the number of examples.	Toy mean is 6.0
T04 **mean_squared_error** **mean_absolute_error**	C07	error = prediction − target. MSE = average of error^2. MAE = average of |error|.	Toy MSE is 12.5 Toy MAE is 3.5
T05 **gradients**	C08	For each example: add 2 × error × features[j] to dw[j], for every j. Then add 2 × error to db **once**. (Dividing by the number of examples is already written.)	Toy: dw = −11, db = −7, plus a two-feature check
T06 **train**	C09	Update every weight and the bias: new value = old value − learning_rate × gradient	Toy model learns; 2,001 loss values
T07 **estimate_mpg**	C14	Make the features using the frozen model's mode, then call predict with its weights and bias. Do **not** call train.	All four app checks run

Stuck? Check in this order

1. Did you run every cell above this one, in order?
2. Are the inputs scaled (divided by 1000, 5 and 8)?
3. Is error = prediction − target, and not the other way round?
4. Are you **subtracting** the gradient, not adding it?
5. Change one thing at a time, then run again.

> **Tip:** the Rent Predictor used the same ideas: predict, mean, MSE, gradients and the update rule. Look at how you solved them there, then think about what is different here.

Step by step

Step	Cells	Do this	Answer sheet
1 Explore the data **40 min**	C01–C04	Fill in the C01 ✏ cell. Run C02–C03 and look at the plot. Do T01.	Q1–Q6
2 Predict and set a baseline **30 min**	C05–C06	Do T02 and T03. The baseline always guesses the training average.	Q7–Q8
3 Measure error and gradients **45 min**	C07–C08	Do T04 and T05. One update by hand: w goes from 1 to 2.1, b from 1 to 1.7, and MSE drops from 12.5 to 1.325.	Q9–Q11
4 Train Model A **35 min**	C09	Do T06. Model A uses mass only, learning rate 0.05 and 2,000 updates. Look at both plots.	Q12–Q13
5 Compare and choose **35 min**	C10–C11	Write your predictions in the C10 ✏ cell, **then** run the experiment. Run C11 and write your choice in its ✏ cell.	Q14–Q16
6 Test and study mistakes **30 min**	C12–C13	Run C12 once. Study the three biggest mistakes in C13.	Q17–Q19
7 Build your predictor **30 min**	C14	Do T07. The checks try two normal cars, one impossible car and one invalid input.	Q20
8 Wrap up **30 min**	C15	Finish the answer sheet and practise your demo.	Q21–Q22

The four candidates in step 5

Candidate	Inputs	Learning rate	Updates
Baseline	None. Always guesses the training average.	–	0
A	Mass	0.05	2,000
B	Mass, engine size, cylinders	0.05	2,000
C	Mass, engine size, cylinders	0.01	2,000

A vs B changes only the inputs. B vs C changes only the learning rate. **The lowest validation MAE wins.** If two are exactly tied, the earlier one wins.

Reading your results

- **MAE** is the average size of the mistakes, in MPG. An MAE of 3 MPG does *not* mean every prediction is within 3 MPG. Do not turn it into an "accuracy %".
- A lower **training** loss does not prove the model works on new cars. That is what validation and test are for.
- A model weight shows a pattern in the data, not proof that one thing causes another. Interpret Model A's weight. Model B's three weights are not reliable on their own, because mass, engine size and cylinders rise together.
- For each big mistake, suggest a possible reason and call it a **hypothesis**: a guess you could test, not a proven fact.

How you are marked

Criterion	Points	What earns the points
Problem and data	15	Correct inputs, target and units; data source and time period; MPG never used as an input. (Q1–Q6)
Prediction and baseline	15	T01–T03 work; hand calculation of a prediction. (Q7–Q8)
Loss and gradients	20	T04–T05 work; toy calculations shown and explained. (Q9–Q11)
Training and experiment	15	T06 works; loss plot; predictions written before running; comparison explained. (Q12–Q15)
Evaluation and mistakes	20	Model chosen with validation before testing; honest test result; three mistakes and limits. (Q16–Q19, Q22)
Code and predictor	10	Notebook runs top to bottom; T07 and the app checks work. (Q20)
Demo and ownership	5	Clear 3-minute demo; each student explains one function and one result without notes. (Q21, contribution table)
Total	100	No bonus points for extra complexity or a lucky test score. An honest result with big errors can still earn full marks.

Before you submit

- ☐ In Jupyter, choose **Kernel → Restart & Run All**. It must finish with no errors.
- ☐ All three ✏ YOUR ANSWER cells are filled in.
- ☐ Every answer-sheet question is answered, and the numbers match your notebook.
- ☐ You did not change the data or move cars between groups.
- ☐ Every group member has filled in their row of the contribution table on the answer sheet.
- ☐ You can explain one function and one result without reading notes.

Optional extras (no marks on their own)

- **Curved input:** add $(\text{mass}/1000 - 1.4)^2$ as a second input next to mass/1000, and compare it on validation. Do this **before** C12. Subtracting 1.4 first helps training finish within 2,000 updates.
- **Save your model (C16):** save the learned numbers to a file, reload them and check that the prediction is the same.

Data source: R. Quinlan (1993), *Auto MPG*, UCI Machine Learning Repository, doi.org/10.24432/C5859H, licence CC BY 4.0. This classroom subset uses source rows 1, 3, 5, ..., 397. Weight was converted from pounds (× 0.45359237) and engine size from cubic inches (× 0.016387064). Cite the source in your answer sheet; these are historical records, not measurements you collected.